BÀI TẬP THỰC HÀNH
XÂY DỰNG HỆ THỐNG GỢI Ý VỚI PYSPARK

1. Mục tiêu thực hành

- Phân biệt Content-Based Filtering và Collaborative Filtering.
- Phân biệt explicit ratings và implicit feedback.
- Hiểu ma trận User-Item, sparsity và trực giác Matrix Factorization.
- Hiểu ý nghĩa latent factors và tham số rank trong ALS.
- Chuẩn bị dữ liệu đúng định dạng userId - movieId - rating cho Spark ALS.
- Huấn luyện mô hình ALS với PySpark.
- Đánh giá dự đoán explicit ratings bằng RMSE.
- Tuning ALS bằng ParamGridBuilder và CrossValidator.
- Sinh Top-N recommendations cho user và item.
- Hiểu cold-start problem và cách xử lý cơ bản.
- Làm quen với implicit ALS và tham số alpha.
- So sánh ALS với các baseline đơn giản như popularity và cosine similarity.

2. Chuẩn bị

- Python 3.x.
- Jupyter Notebook/JupyterLab hoặc VS Code.
- Java tương thích với phiên bản PySpark.
- PySpark, Pandas, NumPy, scikit-learn và SciPy.
- Notebook TH_Building_Recommendation_Engines_with_PySpark.ipynb.
- Tệp dữ liệu data/user_ratings.csv.

```
pip install pyspark pandas numpy scikit-learn scipy matplotlib
```

3. Nội dung thực hành

Recommendation Engine là hệ thống dự đoán các item mà một user có khả năng quan tâm dựa trên rating, lịch sử tương tác hoặc đặc trưng của item.

User behavior / Ratings → User-Item Matrix → Recommendation Model → Predicted Score → Top-N

Các ứng dụng phổ biến gồm phim, nhạc, sản phẩm thương mại điện tử, bài báo, khóa học và nội dung số.

1. Content-Based và Collaborative Filtering

Content-Based Filtering

Dựa trên đặc trưng của item, ví dụ genre, actor, language, year. Nếu user thích một item thì hệ thống tìm các item có đặc trưng tương tự.

Collaborative Filtering

Dựa trên mẫu sở thích/hành vi tương tự giữa các user hoặc item. ALS trong Spark thuộc nhóm collaborative filtering dựa trên matrix factorization.

Tiêu chí	Content-Based	Collaborative Filtering

Nguồn dữ liệu	Feature của item	User-item interactions
Metadata item	Thường cần	Không bắt buộc
Khám phá sở thích cộng đồng	Hạn chế	Tốt
Cold-start	Có thể tận dụng metadata	Khó với user/item mới

2. Explicit và Implicit Ratings

Explicit Ratings

- 1-5 sao
- like/dislike
- score trực tiếp do user cung cấp

Implicit Feedback

- số lượt xem/nghe
- click
- thời gian xem
- mua hàng
- add-to-cart

Lưu ý: Implicit feedback thường biểu diễn mức độ tương tác/confidence, không đồng nghĩa trực tiếp với mức độ thích.

3. User-Item Matrix và Matrix Factorization

Ma trận rating có hàng là user, cột là item, ô là rating. Phần lớn các ô thường trống nên dữ liệu recommendation có sparsity cao.

```
# Minh họa khái niệm
#       Movie1 Movie2 Movie3
# User1    5     4     ?
# User2    4     ?     5
# User3    1     2     5
```

Matrix Factorization

Ý tưởng cốt lõi:

$$R \approx U \times V$$

- R: ma trận user-item ban đầu.
- U: latent factors của user.
- V: latent factors của item.
- rank = k: số latent factors.

Trực giác ALS

ALS lặp giữa hai bước: giữ user factors cố định để tối ưu item factors, rồi giữ item factors cố định để tối ưu user factors. Quá trình lặp giúp lỗi dự đoán giảm dần.

4. Chuẩn bị dữ liệu cho Spark ALS

Spark ALS yêu cầu dữ liệu dạng long/row-based với ba cột số:

```
userId | movieId | rating
```

Khởi tạo Spark và đọc dữ liệu

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = (
    SparkSession.builder
    .appName("RecommendationEnginesWithPySpark")
    .master("local[*]")
    .getOrCreate()
)

ratings = spark.read.csv(
    "data/user_ratings.csv",
    header=True,
    inferSchema=True
)

ratings.show(10, truncate=False)
ratings.printSchema()
```

Chuẩn hóa cột cho ALS

```
als_data = (
    ratings
    .select(
        F.col("userId").cast("int").alias("userId"),
        F.col("movieId").cast("int").alias("movieId"),
        F.col("rating").cast("float").alias("rating")
    )
    .dropna()
)

als_data.printSchema()
```

Lưu ý: userCol và itemCol của ALS phải là kiểu số nguyên. Nếu ID ban đầu là chuỗi, cần ánh xạ sang ID integer rồi join trở lại dữ liệu.

5. Khảo sát dữ liệu và Sparsity

Thống kê cơ bản

```
print("Ratings:", ratings.count())
print("Users  :", ratings.select("userId").distinct().count())
print("Movies :", ratings.select("movieId").distinct().count())
```

Số rating theo user

```
user_counts = ratings.groupBy("userId").count()

user_counts.select(
    F.min("count").alias("min_ratings"),
    F.max("count").alias("max_ratings"),
    F.avg("count").alias("avg_ratings")
).show()
```

Tính sparsity

```
numerator = ratings.count()
num_users = ratings.select("userId").distinct().count()
num_movies = ratings.select("movieId").distinct().count()

denominator = num_users * num_movies
density = numerator / denominator
sparsity = 1 - density

print("Density :", density)
print("Sparsity:", sparsity)
```

Lưu ý: Sparsity gần 1 nghĩa là phần lớn cặp user-item chưa có rating; đây là tình huống phổ biến của recommendation systems.

6. ALS và các tham số

Tham số	Ý nghĩa	Gợi ý
rank	Số latent factors	Tăng rank làm model linh hoạt hơn nhưng tốn tài nguyên.
maxIter	Số lần lặp ALS	Tăng nếu model chưa hội tụ.
regParam	Regularization λ	Giúp hạn chế overfitting.
alpha	Mức confidence của implicit feedback	Chủ yếu dùng khi implicitPrefs=True.
nonnegative	Ép latent factors không âm	Có thể hữu ích cho interpretability.
coldStartStrategy	Cách xử lý prediction NaN	Dùng 'drop' khi đánh giá.
implicitPrefs	Explicit hay implicit ALS	False cho rating trực tiếp.

7. Xây dựng ALS cơ bản

```
from pyspark.ml.recommendation import ALS

training_data, test_data = als_data.randomSplit(
    [0.8, 0.2],
    seed=42
)

als = ALS(
    userCol="userId",
    itemCol="movieId",
    ratingCol="rating",
    rank=20,
    maxIter=15,
    regParam=0.1,
    nonnegative=True,
    coldStartStrategy="drop",
    implicitPrefs=False,
    seed=42
)

model = als.fit(training_data)
predictions = model.transform(test_data)
```

```
predictions.show(10)
```

Trong workflow chuẩn: `fit()` chỉ dùng training data; test data chỉ dùng để đánh giá cuối cùng.

8. Đánh giá bằng RMSE

Với explicit ratings, RMSE đo độ lệch giữa rating dự đoán và rating thực.

```
from pyspark.ml.evaluation import RegressionEvaluator

evaluator = RegressionEvaluator(
    metricName="rmse",
    labelCol="rating",
    predictionCol="prediction"
)

rmse = evaluator.evaluate(predictions)
print("RMSE =", rmse)
```

8.1. Xem các prediction có sai số lớn

```
(
    predictions
    .withColumn("error", F.col("prediction") - F.col("rating"))
    .withColumn("abs_error", F.abs("error"))
    .orderBy(F.desc("abs_error"))
    .show(20)
)
```

Lưu ý: RMSE phù hợp để đánh giá dự đoán rating, nhưng chưa đủ để đánh giá chất lượng ranking Top-N trong hệ thống thực tế.

9. Cross Validation và Grid Search

Slide sử dụng ParamGridBuilder và CrossValidator để thử nhiều bộ hyperparameter rồi chọn best model.

```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator

als_cv = ALS(
    userCol="userId",
    itemCol="movieId",
    ratingCol="rating",
    coldStartStrategy="drop",
    nonnegative=True,
    implicitPrefs=False,
    seed=42
)

param_grid = (
    ParamGridBuilder()
    .addGrid(als_cv.rank, [10, 20, 40])
    .addGrid(als_cv.maxIter, [10, 20])
    .addGrid(als_cv.regParam, [0.05, 0.1, 0.2])
    .build()
)

cv = CrossValidator(
    estimator=als_cv,
```

```
    estimatorParamMaps=param_grid,
    evaluator=evaluator,
    numFolds=3,
    seed=42,
    parallelism=2
)

cv_model = cv.fit(training_data)
best_model = cv_model.bestModel
```

Đánh giá best model

```
best_predictions = best_model.transform(test_data)
best_rmse = evaluator.evaluate(best_predictions)

print("Best RMSE:", best_rmse)
print("Best rank:", best_model.rank)
print("Best maxIter:", best_model._java_obj.parent().getMaxIter())
print("Best regParam:", best_model._java_obj.parent().getRegParam())
```

Lưu ý: Grid trong slide lớn hơn; trong thực hành nên dùng grid vừa phải để phù hợp máy cá nhân/Colab.

10. Sinh Top-N Recommendation

Top-5 phim cho mỗi user

```
user_recs = best_model.recommendForAllUsers(5)
user_recs.show(5, truncate=False)
```

Nối movie title

```
movie_lookup = ratings.select(
    "movieId", "title"
).dropDuplicates(["movieId"])

exploded_recs = (
    user_recs
    .select("userId", F.explode("recommendations").alias("rec"))
    .select(
        "userId",
        F.col("rec.movieId").alias("movieId"),
        F.col("rec.rating").alias("predicted_rating")
    )
    .join(movie_lookup, "movieId", "left")
)

exploded_recs.show(30, truncate=False)
```

Recommendation cho một user

```
sample_user = ratings.select("userId").first()["userId"]

user_subset = spark.createDataFrame(
    [(int(sample_user),)],
    ["userId"]
)

best_model.recommendForUserSubset(
    user_subset,
```

```
    10
).show(truncate=False)
```

11. Cold Start

ALS không thể tạo latent factors cho user hoặc item chưa xuất hiện trong training data. Đây là cold-start problem.

- User mới: chưa có lịch sử.
- Item mới: chưa có rating.
- Trong đánh giá: coldStartStrategy='drop' giúp loại prediction NaN.
- Trong production: có thể kết hợp popularity, content-based hoặc hybrid.

12. Implicit Feedback với ALS

Khi dữ liệu là lượt xem/lượt nghe/click, dùng `implicitPrefs=True`. `alpha` biểu diễn mức confidence.

```
implicit_data = (
    ratings
    .withColumn(
        "preference",
        F.when(F.col("rating") >= 4.0, 1.0).otherwise(0.0)
    )
    .select(
        F.col("userId").cast("int"),
        F.col("movieId").cast("int"),
        F.col("preference").cast("float")
    )
)

implicit_als = ALS(
    userCol="userId",
    itemCol="movieId",
    ratingCol="preference",
    rank=20,
    maxIter=10,
    regParam=0.1,
    alpha=20,
    implicitPrefs=True,
    coldStartStrategy="drop",
    nonnegative=True,
    seed=17
)

implicit_model = implicit_als.fit(implicit_data)
```

Lưu ý: Với implicit feedback, mục tiêu thường là ranking hơn là dự đoán rating chính xác. Các metric như Precision@K, Recall@K, NDCG hoặc đánh giá online thường phù hợp hơn RMSE.

13. Mở rộng từ code đính kèm

Popularity Baseline

```
movie_stats = (
    ratings
    .groupBy("movieId", "title")
```

```
    .agg(
        F.count("*").alias("num_ratings"),
        F.avg("rating").alias("avg_rating")
    )
)

popular_movies = (
    movie_stats
    .filter(F.col("num_ratings") >= 100)
    .orderBy(F.desc("avg_rating"))
)

popular_movies.show(20, truncate=False)
```

Baseline popularity rất đơn giản nhưng cần thiết để biết ALS có thật sự tốt hơn một chiến lược đơn giản hay không.

Co-occurrence / Movie Pairs

Với từng user, tạo các cặp phim đã cùng được rating/xem. Cặp xuất hiện nhiều lần có thể dùng làm recommendation đơn giản.

```
from itertools import permutations

def create_pairs(movie_series):
    values = movie_series.drop_duplicates().values
    return list(permutations(values, 2))
```

Item-Item Cosine Similarity

```
from sklearn.metrics.pairwise import cosine_similarity

# pivot: user × movie
pivot = ratings_pd.pivot_table(
    index="userId",
    columns="title",
    values="rating"
)

# center theo user rồi fill 0
centered = pivot.sub(pivot.mean(axis=1), axis=0).fillna(0)

item_similarity = cosine_similarity(centered.T)
```

Cosine similarity cho phép tìm các item có pattern rating tương tự. Đây là cách tốt để so sánh với ALS.

4. Bài tập thêm

Xây dựng một hệ thống gợi ý phim hoàn chỉnh từ `user_ratings.csv`.

4.1. Yêu cầu

1. Khảo sát số users, movies, ratings và sparsity.
2. Chia train/test theo tỷ lệ 80/20.
3. Xây dựng ALS baseline.
4. Đánh giá RMSE.
5. Tuning rank, maxIter và regParam bằng Cross Validation.
6. Chọn best model.
7. Sinh Top-10 recommendations cho ít nhất 5 users.
8. Nối movieId với title.
9. So sánh ALS với popularity baseline hoặc item-item cosine similarity.
10. Phân tích cold-start problem.
11. Nhận xét ưu/nhược điểm của mô hình.

4.2. Bảng kết quả đề nghị

Mô hình	rank	regParam	maxIter	RMSE

5. Bài tập mở rộng

Bài 1 - Recommendation Fundamentals

- Giải thích Content-Based và Collaborative Filtering.
- Nêu ví dụ explicit và implicit feedback.

Bài 2 - Sparsity

- Tính density và sparsity của user_ratings.csv.
- Giải thích vì sao recommendation data thường sparse.

Bài 3 - ALS Baseline

- Dùng rank=10, maxIter=10, regParam=0.1.
- Tính RMSE trên test set.

Bài 4 - Rank

- Thử rank = 5, 10, 20, 40, 80.
- Lập bảng Rank vs RMSE.

Bài 5 - Regularization

- Thử regParam = 0.01, 0.05, 0.1, 0.5, 1.0.
- Nhận xét xu hướng RMSE.

Bài 6 - Cross Validation

- Grid rank=[10,20,40], regParam=[0.05,0.1,0.2].
- Dùng 3-fold CV và tìm best model.

Bài 7 - Top-N Recommendation

- Chọn 10 users bất kỳ.
- Sinh Top-5 recommendation và hiển thị title.

Bài 8 - Popularity Baseline

- Chỉ xét phim có ít nhất 100 ratings.
- So sánh Top-N với ALS.

Bài 9 - Similar Movies

- Dùng cosine similarity.
- Tìm 10 phim tương tự Toy Story (1995).

Bài 10 - Implicit ALS

- Chuyển rating >= 4 thành positive interaction.
- Thử alpha = 10, 20, 40.

Bài 11 - Cold Start

- Tạo user mới và movie mới.
- Quan sát prediction và đề xuất cách xử lý.

6. Câu hỏi ôn tập và yêu cầu nộp bài

6.1. Câu hỏi ôn tập

1. Matrix factorization dùng để làm gì trong recommendation system?
2. Latent factor là gì?
3. rank trong ALS có ý nghĩa gì?
4. regParam ảnh hưởng đến mô hình như thế nào?
5. Tại sao cần coldStartStrategy='drop' khi đánh giá?
6. RMSE nhỏ hơn có luôn đồng nghĩa recommendation tốt hơn không?
7. Explicit ratings và implicit feedback khác nhau thế nào?
8. alpha được dùng trong trường hợp nào?
9. Tại sao nên có popularity baseline?
10. Cross Validation khác với đánh giá trên test set như thế nào?

6.2. Yêu cầu nộp bài

- Notebook đã chạy đầy đủ các phần được yêu cầu.
- Bảng thống kê dữ liệu và sparsity.

- Bảng kết quả các cấu hình ALS.
- Best hyperparameters và RMSE trên test set.
- Top-N recommendations có movie title.
- So sánh ít nhất một baseline.
- Nhận xét về cold start, sparsity và giới hạn của RMSE.
- Có kết luận ngắn về mô hình được lựa chọn.

Tổng kết

User-Item Matrix → Matrix Factorization → ALS → RMSE → Cross Validation → Top-N Recommendation

Sau bài thực hành, sinh viên cần hiểu không chỉ cách gọi API ALS mà còn mối liên hệ giữa ma trận rating, latent factors, hyperparameter tuning, sparsity và chất lượng recommendation.

Ghi nhớ: Trong hệ thống thực tế, đánh giá tốt nhất cuối cùng là phản ứng của người dùng với recommendation, chẳng hạn click, xem, mua hoặc conversion.

---o0o---

(Hết)